Backend · Cambios al modelo de datos

Oct 12, 2026 · @German

Esto es una propuesta de Backend: el modelo de Modelado de Datos (Mateo y Valentín) no se modificó. Con la información disponible, Backend armó solo el script de base que su código necesita para funcionar y probarse. Mantiene las 4 tablas y las 3 decisiones de Datos (fen_before/fen_after, source USER/SEED, ON DELETE CASCADE) y suma 9 columnas y 1 vista. Datos decide si lo adopta, lo cambia o lo reemplaza, y Backend ajusta su código a lo que decidan.

Qué propone Backend

Tabla	Cambio	Para qué	Origen
games	+ status (IN_PROGRESS, PAUSED, FINISHED)	Pausar, reanudar y filtrar el historial por pendientes	RN-01
games	+ end_reason (CHECKMATE, STALEMATE, THREEFOLD_REPETITION, FIFTY_MOVES, INSUFFICIENT_MATERIAL, RESIGNATION)	Mostrar por qué terminó la partida	CU-06, RN-03
games	+ white_alias, + black_alias	Guardar el nombre de quien juega cada color	Corrección C4 del profesor
games	+ 3 restricciones CHECK	Una partida FINISHED siempre tiene resultado y fecha de fin; contra la máquina siempre tiene dificultad	Integridad
moves	+ uci (ej. e2e4, e7e8q)	Formato del dataset de Simulación (FEN -> jugada)	Simulación

Tabla	Cambio	Para qué	Origen
moves	+ fen_before_key (FEN de 4 campos, calculado) + índice	Que la IA encuentre la misma posición aunque ocurra en otra jugada	Arquitectura 6.2
moves	+ UNIQUE(game_id, move_number)	Que dos pedidos simultáneos no guarden la misma jugada dos veces	Integridad
difficulty_profiles	+ elo_min, + elo_max, UNIQUE(engine_type, name)	Rango ELO que muestra la pantalla y recibe la IA	Requisitos (pestaña 01)
(nueva)	Vista v_training_moves	La IA entrena solo con partidas FINISHED	RN-01, Arquitectura 7

Las partidas importadas de Lichess (source = SEED) pueden quedar sin end_reason, porque muchas terminaron por tiempo.

Perfiles cargados de entrada (6 filas): Fácil, Medio y Difícil para STOCKFISH (Skill Level 3, 10 y 20) y para AI, con ELO 800-1200, 1300-1700 y 1800+.

Cómo crear la base

El modelo físico completo está en `backend/db/migrations/001_init.sql` y se probó contra PostgreSQL 16. Crea las 4 tablas, los índices, la vista y los 6 perfiles.

1. Crear una base vacía en PostgreSQL.
2. Poner su dirección en `DATABASE_URL` dentro de `backend/.env`.
3. Correr `pnpm migrate` en la carpeta `backend`. Registra qué migraciones ya corrió, así que se puede repetir sin romper nada.

No correrlo contra la base compartida de Supabase hasta acordarlo con GC y Modelado de Datos: si ya hay tablas con esos nombres y otras columnas, la migración las saltea y el backend falla.

Pendiente con Modelado de Datos

- [ ] Mateo y Valentín: incorporar estos cambios al modelo entidad-relación y al modelo físico (o proponer otros y Backend ajusta el código).
- [ ] Corregir en su pestaña que las partidas son "inmutables": games se actualiza en cada jugada; lo inmutable son las jugadas (moves).
- [ ] Justificar o conectar ai_model_versions. Opción: guardar en games qué versión del modelo jugó cada partida contra la IA.
- [ ] Decidir con GC si la base es PostgreSQL local o Supabase, y crearla.
- [ ] Confirmar con el profesor si el ELO se usa o es solo informativo (si cambia, solo cambian los valores de difficulty_profiles).